

Patron de Conception : Composite

Définition

Le **patron Composite** est un patron structurel qui permet de **composer des objets sous forme d'une structure arborescente** et de les traiter de manière uniforme. Il permet de manipuler **de façon identique** des objets simples et des objets composites.

Exemple d'Application

Dans une application de **vente de PC personnalisés**, les utilisateurs peuvent configurer un ordinateur en choisissant différents composants :

- **Objets simples** : CPU, barrette de mémoire
- **Objets composites** : Carte mère contenant plusieurs cartes additionnelles

L'application doit être capable de :

- ✓ **Calculer le prix total** de l'ordinateur en additionnant les prix des composants
- ✓ **Valider la configuration** en vérifiant que tous les composants sont compatibles

1. Première Version du Code (Sans Composite)

Code initial

Classe **Computer**

```
public class Computer {  
    private double price = 0;  
    private CPU cpu;  
    private Memory[] memories;
```

```

private Board[] boards;

public Computer(CPU cpu, Memory[] memories, Board[] boards) { // (1)
Manque de flexibilité
    this.cpu = cpu;
    this.memories = memories;
    this.boards = boards;
    calculatePrice();
}

private void calculatePrice() {
    price = cpu.getPrice(); // (2) Incohérence des méthodes de prix
    for (Memory memory : memories) {
        price += memory.getMemoryPrice();
    }
    for (Board board : boards) {
        price += board.getPrice();
    }
}

public double getPrice() {
    return price;
}

public boolean validateConfiguration() {
    if (!cpu.validateConfiguration()) return false;
    for (Memory memory : memories) {
        if (!memory.validateMemory()) return false;
    }
    for (Board board : boards) {
        if (!board.validateConfiguration()) return false;
    }
    return true;
}
}

```

Classe Board

```

enum BoardType { Graphic, Network }

class Board {
    private String name;
    private BoardType type;
    private Memory[] memories;
    private CPU cpu;

    public Board(String name, BoardType type, CPU cpu, Memory[] memories) {
        this.name = name;
        this.type = type;
        this.cpu = cpu;
        this.memories = memories;
    }

    public double getPrice() {
        double totalPrice = cpu.getPrice();
        for (Memory memory : memories) {
            totalPrice += memory.getMemoryPrice();
        }
        return totalPrice;
    }

    public boolean validateConfiguration() {
        return true; // Simulation
    }
}

```

Classe **Memory**

```

public enum MemoryType { DDR4, GDDR5 }

public class Memory {
    private double price;
    private String name;
    private MemoryType type;
}

```

```

    public Memory(String name, MemoryType type, double price) {
        this.name = name;
        this.type = type;
        this.price = price;
    }

    public double getMemoryPrice() {
        return price;
    }

    public boolean validateMemory() {
        return true;
    }
}

```

Classe CPU

```

public class CPU {
    private double price;
    private String name;

    public CPU(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public double getPrice() {
        return price;
    }

    public boolean validateConfiguration() {
        return true;
    }
}

```

Problèmes du Code Initial

❌ **Manque de flexibilité** : `Computer` prend un `CPU`, un tableau `Memory[]`, et un tableau `Board[]`. Ajouter un nouveau type de composant nécessiterait une modification directe du constructeur.

❌ **Incohérence des méthodes de prix** : `getMemoryPrice()` pour `Memory`, `getPrice()` pour `CPU`, `Board` et `Computer`.

❌ **Difficulté d'extension** : Ajouter un nouveau composant nécessiterait de modifier la méthode `calculatePrice()`.

2. Solution : Utilisation du Patron Composite

Principe de la solution

- **Création d'une interface** `Component` qui représente un élément du PC.
- **Création d'une classe** `CompositeComponent` qui gère des sous-composants.
- **Refonte des classes** `Computer` et `Board` pour qu'elles deviennent des composants composites.

Interface `Component`

Tous les composants du PC (simples et composites) doivent implémenter cette interface.

```
public interface Component {  
    double getPrice();  
    boolean validateConfiguration();  
}
```

Classe `CompositeComponent`

Gère une liste de sous-composants.

```
import java.util.ArrayList;  
import java.util.List;  
  
public abstract class CompositeComponent implements Component {
```

```

private final String name;
private final double price;
private final List<Component> subComponents = new ArrayList<>();

public CompositeComponent(String name, double price) {
    this.name = name;
    this.price = price;
}

@Override
public double getPrice() {
    double totalPrice = price;
    for (Component component : subComponents) {
        totalPrice += component.getPrice();
    }
    return totalPrice;
}

@Override
public boolean validateConfiguration() {
    for (Component component : subComponents) {
        if (!component.validateConfiguration()) return false;
    }
    return true;
}

public void addComponent(Component component) {
    subComponents.add(component);
}
}

```

Classe **Computer**

Devient un **composant composite** en héritant de **CompositeComponent**.

```

public class Computer extends CompositeComponent {
    public Computer(String name, double price) {
        super(name, price);
    }
}

```

```
}  
}
```

Classe **Board**

Devient un **composant composite** en héritant de **CompositeComponent**.

```
public class Board extends CompositeComponent {  
    private BoardType type;  
  
    public Board(String name, BoardType type, double price) {  
        super(name, price);  
        this.type = type;  
    }  
}
```

Code du **main**

Démonstration du fonctionnement du patron Composite.

```
public static void main(String[] args) {  
    CPU cpu = new CPU("Intel i9", 400);  
    CPU graphicCardCpu = new CPU("NVIDIA GPU", 250);  
  
    Memory[] computerMemories = {  
        new Memory("8GB DDR4", MemoryType.DDR4, 80),  
        new Memory("16GB DDR4", MemoryType.DDR4, 150)  
    };  
  
    Board graphicCard = new Board("Graphic Card", BoardType.Graphic, 200);  
    graphicCard.addComponent(new Memory("4GB GDDR5", MemoryType.GDDR5, 200));  
    graphicCard.addComponent(graphicCardCpu);  
  
    Computer computer = new Computer("Custom PC", 500);  
    computer.addComponent(cpu);  
    for (Memory memory : computerMemories) {
```

```
        computer.addComponent(memory);
    }
    computer.addComponent(graphicCard);

    if (computer.validateConfiguration()) {
        System.out.println("Configuration ok");
        System.out.println("Price : " + computer.getPrice());
    } else {
        System.out.println("Invalid Configuration");
    }
}
```

Conclusion

- ✓ **Ajout facile de nouveaux composants** grâce à une structure flexible
- ✓ **Méthodes uniformisées** (`getPrice()` , `validateConfiguration()`)
- ✓ **Extensibilité sans modifier le code existant**

Ce modèle **facilite la gestion des composants** et **rend le code plus flexible et évolutif**.